

Applied Deep Learning — Exam Study Guide (Detailed)

Curated Q&A based on Prof. Dr. Lars Ackermann-Igl's slides

Contents

1	How to use this guide	3
2	1. Threshold Logic Units (TLUs)	4
2.1	Q1. What is a TLU, and how does it decide its output? (Most fundamental definition question)	4
2.2	Q2. Why does a TLU need <i>weights</i> at all — couldn't it just add up the raw inputs?	4
2.3	Q3. What is “linear separability,” and why can a single TLU only solve linearly separable problems? (Very classic exam question — usually tested with XOR)	4
2.4	Q4. How is a TLU trained? Explain the Delta Rule (Widrow–Hoff Procedure) in detail.	5
2.5	Q5. What is the “bias trick” (turning the threshold into a weight), and why do we bother doing it?	6
2.6	Q6. A single TLU can't solve XOR/bi-implication. How does stacking TLUs into a network fix this? Walk through the bi-implication example.	6
3	2. From TLUs to Real Neurons (Activation Functions & Bias)	7
3.1	Q7. Why do we replace the hard step function with a smooth activation function (sigmoid, tanh, ReLU, etc.)?	7
3.2	Q8. Compare the four common activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU. (Classic “compare and contrast” exam question)	7
3.3	Q9. What is the “bias” in a general neuron, and what does it actually do geometrically?	7
4	3. Networks, Forward Propagation, and Why We Need Backpropagation	9
4.1	Q10. What is “forward propagation,” and how is the output of a multi-layer network computed?	9
4.2	Q11. Why can't we simply use the Delta Rule to train the hidden layers of a multi-layer network?	9
4.3	Q12. What is Gradient Descent? Explain the intuition and the formula.	9
4.4	Q13. Describe the Backpropagation algorithm step by step, and explain how the “delta” of a hidden neuron is computed differently from an output neuron. (This is the single most important topic — expect a full worked-example question on this)	10
5	4. Problems That Occur During Training	12
5.1	Q14. Explain the Vanishing Gradient problem: what causes it, and why does ReLU help fix it?	12
5.2	Q15. Explain the Exploding Gradient problem.	12
5.3	Q16. Describe the four classic geometric problems with the error surface that make gradient descent hard, and their typical fixes.	12

5.4	Q17. Explain overfitting vs. underfitting, and how the train/test split is used to detect/prevent it.	13
6	Quick-Reference Formula Sheet (for last-minute review)	14

1 How to use this guide

This guide only contains the questions that are genuinely **likely to be asked** in the exam — the core definitions you must be able to explain, the derivations the professor walked through step by step, and the worked numerical examples (because professors love turning those into exam problems with different numbers). Every answer is written in **plain language first**, then backed up with the formula, then closed with a small example so you can see *why* it works, not just *that* it works.

2 1. Threshold Logic Units (TLUs)

2.1 Q1. What is a TLU, and how does it decide its output? (Most fundamental definition question)

Simple idea: A TLU is the simplest possible “neuron.” It looks at several input numbers, multiplies each one by an importance value (a *weight*), adds everything up, and then makes one yes/no decision: if the total is big enough, it fires a 1; otherwise it outputs 0.

Think of it like a bouncer at a club who has a checklist (inputs), gives each item on the checklist a certain number of “importance points” (weights), adds up the total points for a person, and lets them in only if the total crosses a minimum score (the threshold θ).

Formally:

$$y = f(\mathbf{xw}) = \begin{cases} 1, & \text{if } \mathbf{xw} \geq \theta \\ 0, & \text{otherwise} \end{cases} \quad \text{where } \mathbf{xw} = \sum_{i=1}^n w_i x_i$$

This decision function f is called the **Heaviside step function**: it is flat at 0, then suddenly jumps to 1 once the weighted sum reaches the threshold — like a light switch with no “dimming” in between.

Why this matters: TLUs (McCulloch & Pitts, 1943) are the historical starting point of every neural network — every more advanced neuron (the kind we use today) is just this same idea, but with a smoother decision function instead of a hard switch.

2.2 Q2. Why does a TLU need *weights* at all — couldn’t it just add up the raw inputs?

Simple idea: Not all inputs are equally important for making a decision, and weights are exactly how we tell the unit “pay more attention to this one, less to that one.”

Example given in the lecture: For detecting a stop sign, the octagon *shape* of the sign is a much stronger clue than its red *color* (other signs are also red). So the shape input should get a bigger weight than the color input. Similarly, in credit card fraud detection, we don’t know in advance whether the transaction *amount* or the transaction *location* is more suspicious — that’s exactly the kind of thing the weights are supposed to capture, and exactly what training (see Q4) is supposed to discover automatically.

2.3 Q3. What is “linear separability,” and why can a single TLU only solve linearly separable problems? (Very classic exam question — usually tested with XOR)

Simple idea: A TLU’s decision rule, $\sum w_i x_i \geq \theta$, is really just the equation of a straight line (in 2D) or a flat plane/hyperplane (in higher dimensions). The TLU can only ever say “everything on this side of the line is a 1, everything on the other side is a 0.” If you can draw **one straight line** that correctly separates all the “1” points from all the “0” points of a function’s truth table, that function is **linearly separable**, and a single TLU can compute it.

Formal definition: Two sets of points are linearly separable if there exists at least one line/plane/hyperplane such that all points of one set are on one side, and all points of the other set are on the other side.

Examples that work (linearly separable): AND, OR, NAND, NOR, implication. You can literally draw one line on a 2D plot of the 4 input combinations (0,0), (0,1), (1,0), (1,1) that separates the 1-outputs from the 0-outputs.

Examples that fail (NOT linearly separable): XOR and bi-implication (XNOR). For bi-implication ($x_1 \leftrightarrow x_2$), the “true” points are (0,0) and (1,1) (diagonal corners) and the “false” points are (0,1) and (1,0) (the other diagonal). No single straight line can separate two diagonal corners from the other two diagonal corners — try drawing it and you’ll see it’s impossible.

Conclusion: A single TLU is geometrically limited to drawing one straight decision boundary, so it can never compute XOR or XNOR. This single fact is the entire motivation for the next idea: stacking TLUs into a network.

2.4 Q4. How is a TLU trained? Explain the Delta Rule (Widrow–Hoff Procedure) in detail.

Simple idea: We don’t want to guess the weights by hand (that only works visually for 2–3 inputs). Instead, we let the computer learn them automatically by repeating a simple 3-step loop many times: *guess* → *check how wrong you were* → *nudge the weights a little in the direction that reduces the wrongness*.

Step-by-step procedure: 1. Start with random weights and threshold. 2. For a training example with known correct answer o , compute what the TLU actually outputs, y . 3. If y is wrong, nudge every weight a little bit using the rule below. 4. Repeat over and over, for every example, until the errors become small enough.

The formula (this is what you need to memorize):

$$\Delta w_i = \eta (o - y) x_i, \quad w_i^{(new)} = w_i^{(old)} + \Delta w_i$$

- o = the correct/desired output (from training data)
- y = what the TLU currently outputs
- x_i = the corresponding input value
- η (“eta”) = the **learning rate** — a small positive number that controls how big each adjustment step is.

Why it makes sense intuitively: If the TLU’s output y was too small ($o - y > 0$, i.e. it should have fired but didn’t), the rule *increases* the weights of the inputs that were “on” ($x_i = 1$), making the TLU more likely to fire next time for similar inputs. If y was too big ($o - y < 0$), it *decreases* those weights. If the output was already correct ($o = y$), nothing changes — no need to fix what isn’t broken.

Worked example (exactly the lecture’s numbers, exam-style): Training sample: $x_1 = 1, x_2 = 1$, correct answer $o = 1$. Starting weights: $w_0 = -5$ (bias), $w_1 = 2, w_2 = 1$. Learning rate $\eta = 1$.

Step 1 — compute current output:

$$y = f(-5 \cdot 1 + 2 \cdot 1 + 1 \cdot 1) = f(-2) = 0 \quad (\text{wrong! should be } 1)$$

Step 2 — compute the weight changes (since $o - y = 1 - 0 = 1$):

$$\Delta w_0 = 1 \cdot 1 \cdot 1 = 1, \quad \Delta w_1 = 1 \cdot 1 \cdot 1 = 1, \quad \Delta w_2 = 1 \cdot 1 \cdot 1 = 1$$

Step 3 — apply the changes:

$$w_0^{new} = -5 + 1 = -4, \quad w_1^{new} = 2 + 1 = 3, \quad w_2^{new} = 1 + 1 = 2$$

If you re-check the output now, it will be closer to (or equal to) the correct answer. **This is exactly the kind of calculation the exam will ask you to redo with different numbers — practice it.**

2.5 Q5. What is the “bias trick” (turning the threshold into a weight), and why do we bother doing it?

Simple idea: Up until now we had to treat the threshold θ differently from the weights w_i — they needed two separate update rules. That’s annoying. So instead, we do a small mathematical trick: we pretend the threshold is just one more weight, attached to a fake input that is always equal to 1.

The math: The firing condition $\sum_{i=1}^n w_i x_i \geq \theta$ is exactly the same as $\sum_{i=1}^n w_i x_i - \theta \geq 0$. If we now define a new weight $w_0 = -\theta$ and a constant input $x_0 = 1$, then $-\theta = w_0 x_0$, and the condition becomes:

$$\sum_{i=0}^n w_i x_i \geq 0$$

Why it’s useful: Now there is only **one type of parameter** (weights) and only **one update rule** (the Delta Rule applies uniformly to $w_0, w_1, \dots, w_n$, since $x_0 = 1$ always). This little “bias neuron” trick is used everywhere in deep learning — every neuron in every modern network still has this special bias weight.

2.6 Q6. A single TLU can’t solve XOR/bi-implication. How does stacking TLUs into a network fix this? Walk through the bi-implication example.

Simple idea: One TLU can only draw one straight line. But if you connect several TLUs together — using the *outputs* of some TLUs as the *inputs* to another TLU — the combination can carve out much more complicated, bent, or non-convex regions. This is the entire point of having “layers.”

Worked example — Bi-implication $x_1 \leftrightarrow x_2$: We know $x_1 \leftrightarrow x_2 \equiv (x_1 \rightarrow x_2) \wedge (x_2 \rightarrow x_1)$ — i.e., bi-implication is just “implication both ways, combined with AND.” Each of these three pieces (two implications + one AND) is linearly separable on its own, so we can build each with its own TLU:

- **TLU 1** computes $y_1 = x_1 \rightarrow x_2$
- **TLU 2** computes $y_2 = x_2 \rightarrow x_1$
- **TLU 3** (the output layer) takes y_1 and y_2 as its *inputs* and computes $y = y_1 \wedge y_2$

So with $x_1 = 1, x_2 = 0$: TLU 1 gives $y_1 = 0$, TLU 2 gives $y_2 = 1$, and TLU 3 combines them to give $y = 0 \wedge 1 = 0$ — which is the correct bi-implication answer for $(1, 0)$.

The big takeaway: the network’s final answer is “a function of a function” — $y = f(f(x))$. This composition of simple linear pieces is exactly what lets a network represent things a single unit cannot. It also raises the **next big question**: we know the correct final answer (for training TLU 3), but how do we know what y_1 and y_2 *should* have been, since nothing in the training data tells us that directly? This unanswered question is exactly what motivates **backpropagation** (Q11–Q13 below).

3 2. From TLUs to Real Neurons (Activation Functions & Bias)

3.1 Q7. Why do we replace the hard step function with a smooth activation function (sigmoid, tanh, ReLU, etc.)?

Simple idea: The Delta Rule worked fine for single TLUs, but to train *networks* with hidden layers, we'll need a much more powerful method called gradient descent (Q9–Q10), and gradient descent **requires the function to be smooth/differentiable** — meaning it needs a well-defined slope everywhere. The step function has a derivative of 0 almost everywhere and is undefined exactly at the threshold, so it gives **no information at all** about which direction to adjust the weights. We therefore swap it out for an activation function that still looks “switch-like” but is smooth.

A general neuron now looks like:

$$y = f(\mathbf{w} \cdot \mathbf{x}) = f\left(\sum_{i=0}^n w_i x_i\right)$$

where f is one of the activation functions below, instead of the hard step function.

3.2 Q8. Compare the four common activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU. (Classic “compare and contrast” exam question)

Sigmoid: $f(x) = \frac{1}{1 + e^{-x}}$. Squeezes any number into the range $(0, 1)$, which is nice for “probability-like” outputs. **Problems:** for very large or very small x , the curve is almost flat, so the gradient (slope) becomes nearly zero — this is the start of the **vanishing gradient problem** (see Q14). It's also not centered at 0 (always positive), and it's relatively expensive to compute because of the exponential.

Tanh: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. Same S-shape as sigmoid, but squeezes values into $(-1, 1)$ instead, so it is **centered around 0** — slightly nicer for training. **Problem:** it still flattens out at the extremes, so the vanishing-gradient problem is still present, and it's even a bit more expensive to compute than sigmoid.

ReLU (Rectified Linear Unit): $R(x) = x$ if $x \geq 0$, else 0. Extremely cheap to compute (just a comparison), and because its slope is exactly 1 for all positive inputs, it **does not suffer from vanishing gradients** on the active side. **Problem:** if a neuron's input is always negative, its output is always 0 and its gradient is always 0 — it can never “wake up” again. This is called a **dead neuron**.

Leaky ReLU: $R(x) = x$ if $x \geq 0$, else αx (a small slope α , e.g. 0.01, instead of a flat 0). This fixes the dead-neuron problem because even negative inputs still produce a tiny non-zero gradient. **Trade-off:** slightly more complex to compute than plain ReLU, and the vanishing-gradient problem can still partially appear if α is very small.

Side note worth remembering: the lecture explicitly says that having *some* dead neurons in practice is not necessarily a disaster — the network can still work fine overall.

3.3 Q9. What is the “bias” in a general neuron, and what does it actually do geometrically?

Simple idea: The bias is the same trick as Q5 (w_0 paired with a constant input $x_0 = 1$), but now explained in terms of *function fitting* rather than logic. Without a bias, the neuron's activation function is forced to pass through the origin $(0, 0)$ — it can only represent functions that happen to cross zero at input zero. The bias lets the whole function **shift up or down**, so it can fit data that doesn't pass through the origin.

Easy way to remember it: think of the line equation $f(x) = ax + c$. The **weight is the slope** a (how steep the line is), and the **bias is the intercept** c (where the line crosses the y-axis). Without the bias/intercept, you could only ever draw lines through the origin — clearly not flexible enough for real data.

4 3. Networks, Forward Propagation, and Why We Need Backpropagation

4.1 Q10. What is “forward propagation,” and how is the output of a multi-layer network computed?

Simple idea: Forward propagation is simply: feed the input into layer 1, take its output, feed that into layer 2, take *that* output, feed it into layer 3, and so on, until you reach the final output layer. Each layer does the same basic operation: multiply by its weights, then apply the activation function.

$$\vec{h}_1 = f(\vec{x} \cdot W_1), \quad \vec{h}_2 = f(\vec{h}_1 \cdot W_2), \quad \dots, \quad \vec{y} = f(\vec{h}_{last} \cdot W_{out})$$

In implementations (e.g. with GPUs), this is always written as **matrix multiplications**, because that is by far the most efficient way to compute it: $Z = XW + B$ (weighted sum + bias), then $H = f(Z)$ (apply activation element-wise).

Important fact worth quoting in an exam: A neural network with just **one hidden layer** is already a **universal function approximator** — meaning, in theory, it can approximate *any* reasonably well-behaved function, given enough neurons in that hidden layer. More neurons = better approximation.

4.2 Q11. Why can’t we simply use the Delta Rule to train the hidden layers of a multi-layer network?

Simple idea: The Delta Rule needs to know the *correct answer* (o) for the neuron it’s updating, so it can compute $(o - y)$. For the **output layer**, the training data directly gives us this correct answer. But for a **hidden layer**, nobody tells us what the “correct” output of that hidden neuron should have been — the training data only specifies the final answer, not what should happen in the middle of the network.

This is exactly why training multi-layer networks needs a smarter algorithm — and is literally why this whole field is called “**deep learning**.” The algorithm that solves this problem is **backpropagation**.

4.3 Q12. What is Gradient Descent? Explain the intuition and the formula.

Simple idea: Imagine the “total error” of the network as a hilly landscape, where the height at each point represents how wrong the network currently is for a given set of weights. Training means trying to walk downhill on this landscape until you reach a valley (a low-error point = good weights). Gradient descent is the formal recipe for “always step downhill.”

The formula:

$$w := w - \eta \nabla J(w)$$

- $J(w)$ = the error/cost function (how wrong the network currently is)
- $\nabla J(w)$ = the **gradient**: a vector that always points in the direction of *steepest increase* of the error
- Since we want to *decrease* error, we move in the *opposite* direction — hence the minus sign
- η = learning rate, controlling step size

What the gradient’s sign tells you (good thing to memorize): - Positive gradient → increasing the weight increases the error → so we should *decrease* the weight. - Negative gradient → increasing the weight decreases the error → so we should *increase* the weight. - Zero gradient → we’re at a flat spot (could be a minimum, or a plateau — see Q15).

Effect of the learning rate η : too small, and training crawls forward extremely slowly. Too large, and the steps overshoot the valley, bouncing back and forth (or even diverging completely) instead of settling down. A commonly used starting value, assuming the input features are normalized, is $\eta = 0.01$.

4.4 Q13. Describe the Backpropagation algorithm step by step, and explain how the “delta” of a hidden neuron is computed differently from an output neuron. (This is the single most important topic — expect a full worked-example question on this)

Simple idea of the whole algorithm: 1. Start with random weights. 2. Run a forward pass — push an input through the network and see what it outputs. 3. Compare the output to the correct answer to get the **error**. 4. Push that error *backwards* through the network, layer by layer, figuring out how much each individual weight (including ones in hidden layers!) contributed to the mistake. 5. Adjust every weight a little bit using gradient descent, based on that “blame” signal. 6. Repeat steps 2–5 many times until the error is small enough.

Important relationship to remember: *Gradient descent is the method that actually updates the weights; backpropagation is just the clever, efficient procedure for computing the gradients/errors that gradient descent needs* — especially for hidden layers, where there’s no direct target value.

The key formula — computing the “delta” (δ) for any neuron i :

$$\delta_i = \begin{cases} f'(net_i)(o_i - y_i), & \text{if neuron } i \text{ is in the OUTPUT layer} \\ f'(net_i) \sum_k (\delta_k^{(\text{next layer})} \cdot w_{ki}), & \text{if neuron } i \text{ is in a HIDDEN layer} \end{cases}$$

In plain words: - For an **output neuron**, we know the correct answer, so its delta is simply “how far off was I” ($o_i - y_i$), scaled by the local slope of the activation function $f'(net_i)$. - For a **hidden neuron**, since we have no direct target, we instead **borrow the deltas of all the neurons in the next layer** that this hidden neuron feeds into, multiply each by the weight of the connection to it, and add them all up. This is literally “passing the blame backwards” — a hidden neuron’s error-share depends on how much it contributed (through its outgoing weights) to the mistakes of the neurons after it.

For the sigmoid activation, the slope is conveniently $f'(x) = f(x)(1 - f(x))$ — easy to compute once you already have $f(x)$.

The weight-update rule (a direct generalization of the Delta Rule from Q4):

$$\Delta w_{ij} = \eta \delta_i o_j$$

where o_j is simply the output of whichever neuron is feeding *into* neuron i (or the raw input x_j , if i is in the first hidden layer).

Full worked example (the lecture’s exact numbers — practice this with different numbers for the exam):

Network: 2 inputs ($x_1 = 0.7, x_2 = 0.6$) + bias $\rightarrow$ 2 hidden neurons (h_1, h_2) + bias $\rightarrow$ 2 output neurons (o_1, o_2). Sigmoid activation everywhere. Target output: $o^{target} = (0.9, 0.2)$. Learning rate $\eta = 1$.

Forward pass: net input to hidden layer = $(1.16, -0.2) \rightarrow$ hidden outputs $(0.761, 0.45)$. Net input to output layer = $(0.639, 0.2) \rightarrow$ output $(0.655, 0.55)$.

Output error: $(0.9 - 0.655, 0.2 - 0.55) = (0.245, -0.35)$.

Output deltas (multiply error by the local slope $f' = f(1 - f)$):

$$\delta(o_1) = 0.655(1 - 0.655)(0.245) \approx 0.055, \quad \delta(o_2) = 0.55(1 - 0.55)(-0.35) \approx -0.087$$

Hidden deltas (sum of next-layer deltas $\times$ connecting weights, scaled by local slope):

$$\delta(h_1) \approx -0.015, \quad \delta(h_2) \approx 0.010$$

Weight updates ($\Delta w = \eta \delta o_{\text{source}}$): for the output-layer weights, multiply each output-delta by the corresponding hidden-neuron output (and by 1 for the bias); for the hidden-layer weights, multiply each hidden-delta by the corresponding input value x_1, x_2 (and by 1 for the bias).

Curious observation the lecture points out: the *relative size* of the weight changes was much bigger for the output layer (~ 0.84 on average) than for the hidden layer (~ 0.027) — hidden layers learn noticeably **more slowly**. This is your first hint of the vanishing gradient problem, explained next.

5 4. Problems That Occur During Training

5.1 Q14. Explain the Vanishing Gradient problem: what causes it, and why does ReLU help fix it?

Simple idea: In a deep network, the error signal has to travel backwards through many layers during backpropagation. At each layer, the signal gets multiplied by the local slope of the activation function. If that slope is always a number smaller than 1 (which is true for sigmoid and tanh), then multiplying many such small numbers together — one per layer — makes the overall signal shrink **exponentially** the deeper it has to travel. By the time it reaches the very first layers, it can become so close to zero that those layers basically **stop learning** entirely.

Why sigmoid is especially bad: its maximum possible slope is only $f'(x) \leq 0.25$. So after n layers, the gradient reaching the first layer is at most roughly 0.25^n times some constant — for even a modestly deep network, this becomes a tiny number very fast.

Why ReLU helps: ReLU's slope is exactly 1 for any positive input (and 0 for negative input). So along the path where neurons are “active” (positive), the gradient is passed backward completely unchanged — it doesn't shrink at all. The trade-off is the “dead neuron” issue mentioned in Q8, but in practice this is a much smaller price to pay than vanishing gradients.

Other cause: weights initialized with very small absolute values can cause the same multiplicative shrinking effect, for the same reason.

5.2 Q15. Explain the Exploding Gradient problem.

Simple idea: This is the mirror-image problem of vanishing gradients. If the slopes being multiplied together (layer after layer) are consistently **greater than 1**, the gradient instead grows exponentially as it travels backward, becoming huge. This causes training to become unstable — the weight updates can become so large that the network's parameters jump around wildly instead of settling into a good solution, and training may diverge instead of converge.

Causes: activation functions with derivative > 1 combined with deep networks (same chain-rule reasoning as vanishing gradients, just in the opposite direction); weights initialized too large; and learning rates that are too high can make the problem worse (or hide it).

5.3 Q16. Describe the four classic geometric problems with the error surface that make gradient descent hard, and their typical fixes.

Problem	What happens	Fix
Local minima	The error landscape has several “valleys,” and gradient descent can get permanently stuck in a shallow one instead of finding the deepest (global) valley. There's no guaranteed universal solution — in practice we just accept a “good enough” minimum.	Adjust the learning rate / try different starting points

Problem	What happens	Fix
Flat plateaus	A wide, almost-flat region of the error surface gives a near-zero gradient, so training crawls forward extremely slowly even though it hasn't actually converged yet.	Add a momentum term, which keeps some of the previous step's "speed" so the optimizer can coast through flat regions
Steep canyons	A sudden flip from a very strong negative slope to a very strong positive slope (and back) makes the weights oscillate back and forth instead of moving smoothly downhill.	Add a momentum term to smooth out the back-and-forth motion
Skipping good minima	On a very steep slope, the gradient is large, so the step taken is large too — large enough that it can jump clean over a good minimum without ever landing in it.	Adjust (lower) the learning rate

5.4 Q17. Explain overfitting vs. underfitting, and how the train/test split is used to detect/prevent it.

Simple idea: After training, we want to know if the network actually *learned the underlying pattern*, or if it just **memorized** the exact training examples by heart (like memorizing answers to last year's exam questions without understanding the topic). A memorizing network will look great on the training data but fail badly on anything new.

- **Overfitting:** the network fits the training data almost perfectly, but its decision boundary is overly twisted/specific to those exact points, so it performs poorly on new, unseen data of the same type.
- **Underfitting:** the opposite extreme — the network's decision boundary is too simple (e.g. trying to use one straight line to capture something that's clearly more complex), so it performs poorly even on training data.
- **Well-learned:** the boundary captures the *real* underlying pattern, generalizing well to new examples too.

How we check for this: split the available data into a **training set** (used to actually adjust the weights) and a **test/verification set** (data the network never sees during training, used only to check how well it generalizes). A common split mentioned in the lecture is **70% training / 30% testing**, chosen randomly. We consider training successfully finished only when the network performs well on **both** sets — good training performance alone tells us nothing about overfitting.

6 Quick-Reference Formula Sheet (for last-minute review)

- **TLU rule:** $y = 1$ if $\sum w_i x_i \geq \theta$, else 0
- **Bias trick:** $-\theta = w_0 x_0$, with constant input $x_0 = 1$, so the rule becomes $\sum_{i=0}^n w_i x_i \geq 0$
- **Delta Rule:** $\Delta w_i = \eta(o - y)x_i$
- **General neuron:** $y = f(\mathbf{w} \cdot \mathbf{x})$, with smooth/differentiable f
- **Sigmoid:** $f(x) = \frac{1}{1 + e^{-x}}$, derivative $f'(x) = f(x)(1 - f(x))$, max slope = 0.25
- **ReLU:** $f(x) = \max(0, x)$, derivative = 1 if $x > 0$ else 0
- **Gradient descent:** $w := w - \eta \nabla J(w)$
- **Backprop, output neuron:** $\delta_i = f'(net_i)(o_i - y_i)$
- **Backprop, hidden neuron:** $\delta_i = f'(net_i) \sum_k \delta_k w_{ki}$
- **Backprop weight update:** $\Delta w_{ij} = \eta \delta_i o_j$